# App “LA VENTECITA”

# Ejemplo de Diseño TOP-DOWN

Desarrollo de [La ventecita] es una pequeña pulpería como la que nos podemos imaginar en nuestra zona donde habitamos, en esta se venden productos perecederos como queso, leche, granos básicos, verduras etc. La Ventecita se quiere automatizar y cuenta actualmente en múltiples cuadernos de registro:

- lista de clientes,
- proveedores,
- ventas,
- ordenes de compras,
- catálogo de productos,
- vendedores,

Además se puede vender a crédito o al contado. Esta App debe contemplar las siguientes salidas:

lista de clientes,

lista de proveedores,

lista de productos con precios,

productos por vencer,

lista de clientes con créditos,

lista de ventas por período,

listado de órdenes de compra,

10 productos más vendidos,

En este ejemplo se continúa con el proceso aplicando la metodología hasta convertir el resultado a pseudocódigo y luego los pasos para implementar en Python.

## FASE 1: APLICACIÓN DE LA METODOLOGÍA TOP-DOWN

### Paso 1: Definición del Problema General (Nivel 0)

- **Objetivo General del Sistema:** Desarrollar una aplicación de escritorio que permita a "La Ventecita" gestionar eficientemente sus operaciones diarias de compra, venta y control de inventario de productos perecederos, mejorando la toma de decisiones a través de reportes clave.
- **Entradas (Datos de Entrada):**
    - Datos de clientes (nombre, teléfono, dirección, límite de crédito).

- Datos de proveedores (nombre, teléfono, dirección).
- Datos de productos (nombre, categoría, precio de compra, precio de venta, fecha de caducidad, stock).
- Registro de ventas (producto, cantidad, cliente, fecha, tipo de pago: contado/crédito).
- Registro de órdenes de compra (producto, cantidad, proveedor, fecha de pedido, fecha de entrega).

- **Salidas (Resultados Esperados):**
  1. Lista de todos los clientes.
  2. Lista de todos los proveedores.
  3. Catálogo de productos con precios.
  4. Lista de productos por vencer (próximos 7 días).
  5. Lista de clientes con crédito pendiente.
  6. Reporte de ventas por período (fecha inicial y final).
  7. Listado de órdenes de compra realizadas.
  8. Top 10 de los productos más vendidos.

**Paso 2: Identificación de Módulos Principales (Nivel 1)**

Dividimos el sistema en 3 bloques funcionales grandes siguiendo el modelo EPS (Entrada, Procesamiento, Salida).

- **Módulo 1: Entrada de Datos (Gestión de Datos Maestros y Transaccionales)**
  - **Responsabilidad:** Capturar y mantener toda la información que alimenta el sistema.
  - **Submódulos:** Gestión de Clientes, Gestión de Proveedores, Gestión de Productos, Gestión de Ventas, Gestión de Órdenes de Compra.
- **Módulo 2: Procesamiento (Lógica de Negocio y Cálculos)**
  - **Responsabilidad:** Ejecutar la lógica central del negocio. Aquí se procesan los datos de entrada para generar los resultados de salida.
  - **Submódulos:** Control de Inventario (actualizar stock al vender/comprar), Gestión de Créditos (calcular deuda de clientes), Cálculo de Reportes (ventas por período, top de productos).
- **Módulo 3: Salida de Datos (Generación de Reportes)**
  - **Responsabilidad:** Presentar la información procesada de manera clara y útil para el usuario.

- **Submódulos:** Generador de listados (clientes, proveedores), Generador de reportes de inventario (productos por vencer), Generador de reportes financieros (ventas por período, clientes con crédito, top 10).

**Paso 3: Refinamiento Progresivo (Nivel 2 en adelante)**

Ahora, desglosamos cada módulo del Nivel 1 en tareas más específicas.

**A. Refinamiento del Módulo 1: Entrada de Datos**

- **A.1. Gestión de Clientes:**
  - A.1.1. alta_cliente(): Solicitar datos (ID, nombre, teléfono, dirección, límite de crédito) y almacenarlos.
  - A.1.2. buscar_cliente(): Buscar un cliente por ID o nombre.
- **A.2. Gestión de Proveedores:**
  - A.2.1. alta_proveedor(): Solicitar datos (ID, nombre, teléfono, dirección) y almacenarlos.
- **A.3. Gestión de Productos (Catalogo):**
  - A.3.1. alta_producto(): Solicitar datos (ID, nombre, categoría, precio_compra, precio_venta, fecha_caducidad, stock_inicial).
- **A.4. Gestión de Ventas:**
  - A.4.1. registrar_venta():
    - A.4.1.1. Mostrar catálogo de productos.
    - A.4.1.2. Seleccionar producto y cantidad.
    - A.4.1.3. **Decisión:** ¿El cliente es nuevo o existente?
      - Si es nuevo, llamar a alta_cliente().
    - A.4.1.4. **Decisión:** ¿Tipo de pago es 'contado' o 'crédito'?
      - Si es 'crédito', verificar el límite de crédito del cliente.
    - A.4.1.5. Calcular el total de la venta.
    - A.4.1.6. **Proceso repetitivo (Bucle):** Permitir agregar más productos a la misma venta.
    - A.4.1.7. Registrar la venta y actualizar el stock del producto.
- **A.5. Gestión de Órdenes de Compra:**
  - A.5.1. registrar_orden_compra():
    - A.5.1.1. Seleccionar proveedor.

        - A.5.1.2. Seleccionar producto y cantidad.
        - A.5.1.3. Registrar la orden con fecha de pedido y entrega.
        - A.5.1.4. **Proceso repetitivo (Bucle):** Permitir agregar más productos a la misma orden.

**B. Refinamiento del Módulo 2: Procesamiento**

- **B.1. Control de Inventario:**
    - B.1.1. actualizar_stock(producto_id, cantidad, operacion): Aumenta o disminuye el stock de un producto. (Se llama desde registrar_venta y registrar_orden_compra).
- **B.2. Gestión de Créditos:**
    - B.2.1. calcular_deuda_cliente(cliente_id): Suma el total de todas las ventas a crédito no pagadas de un cliente.
- **B.3. Motor de Reportes:**
    - B.3.1. calcular_ventas_por_periodo(fecha_inicio, fecha_fin): Filtra las ventas por fecha y calcula el total.
    - B.3.2. calcular_top_10_productos(): Ordena los productos por cantidad vendida y selecciona los 10 primeros.

**C. Refinamiento del Módulo 3: Salida de Datos**

- **C.1. Generador de Listados:**
    - C.1.1. imprimir_clientes(): Recorre la lista de clientes y la muestra en pantalla.
    - C.1.2. imprimir_proveedores(): Recorre la lista de proveedores y la muestra.
    - C.1.3. imprimir_productos(): Muestra el catálogo con precios.
- **C.2. Generador de Reportes de Inventario:**
    - C.2.1. productos_por_vencer(dias=7): Filtra los productos cuya fecha de caducidad sea menor a la fecha actual + dias. **Decisión:** Imprime la lista si hay productos, si no, muestra un mensaje.
- **C.3. Generador de Reportes Financieros:**
    - C.3.1. imprimir_clientes_con_credito(): Recorre todos los clientes, calcula su deuda con calcular_deuda_cliente() y muestra aquellos con deuda > 0.
    - C.3.2. imprimir_ventas_por_periodo(): Solicita fechas al usuario, llama a calcular_ventas_por_periodo() y muestra el resultado.
    - C.3.3. imprimir_ordenes_compra(): Muestra el listado de todas las órdenes.
    - C.3.4. imprimir_top_10(): Llama a calcular_top_10_productos() y muestra el resultado.

### Paso 4: Verificación de Interdependencias

Revisamos que la salida de un módulo sirva como entrada del siguiente.

1. **Alta de Productos (Módulo 1.A.3)** -> El catálogo de productos generado es la **entrada** para registrar_venta() (Módulo 1.A.4) y registrar_orden_compra() (Módulo 1.A.5).
2. **Alta de Clientes (Módulo 1.A.1)** -> La lista de clientes es la **entrada** para registrar_venta() al asignar la venta a un cliente.
3. registrar_venta() **(Módulo 1.A.4)** -> Actualiza el stock (procesamiento B.1) y genera un registro de venta. Este registro de venta es la **entrada** para calcular_ventas_por_periodo() (B.3.1) y calcular_top_10_productos() (B.3.2).
4. calcular_top_10_productos() **(B.3.2)** -> Su resultado es la **entrada** para imprimir_top_10() (C.3.4).
5. **Gestión de Créditos (B.2)** -> La función calcular_deuda_cliente() necesita los registros de venta (Módulo 1.A.4). Su resultado es la **entrada** para imprimir_clientes_con_credito() (C.3.1).

### Actividade en clase (Tabla de interdependencias):

| Nombre del módulo | Módulo de nivel superior | Código de módulo | Datos entrada | Datos salida | Interdependencias |
|---|---|---|---|---|---|
| Alta de productos | **Gestión de Productos (Catalogo)** | A.3.1 | ID, nombre, categoría, precio_compra, precio_venta, fecha_caducidad, stock_inicial | ID, nombre, categoría, precio_compra, precio_venta, fecha_caducidad, stock_inicial | -Módulo A.4.1: registrar_venta(nombre, precio_venta, stock_inicial)<br><br>-Módulo A.5.1: registrar_orden_compra(nombre, categoría, precio_compra, fecha_caducidad, stock_inicial) |

Se verificó que el flujo de datos es coherente. Las salidas de los módulos de entrada alimentan a los de procesamiento, y las salidas de procesamiento alimentan a los de salida.

---

### FASE 2: CONVERSIÓN A PSEUDOCÓDIGO

Tomamos un módulo complejo, por ejemplo, el **Módulo 1.A.4 (registrar_venta)**, y lo convertimos a pseudocódigo para planificar la lógica antes de codificar.

**Ejemplo Pseudocódigo para** registrar_venta()**:**

```
INICIO registrar_venta()

  // 1. Inicializar variables
```

```
total_venta = 0.0
lista_productos_vendidos = []  // Lista para guardar los items de la venta

// 2. Identificar al Cliente
MOSTRAR "Ingrese ID del cliente:"
LEER id_cliente
cliente = buscar_cliente(id_cliente)

SI cliente es NULO ENTONCES
  MOSTRAR "Cliente no encontrado. ¿Desea registrarlo? (S/N)"
  LEER respuesta
  SI respuesta == 'S' ENTONCES
    cliente = alta_cliente()  // Llama a otro subproceso
  SINO
    MOSTRAR "Venta cancelada."
    TERMINAR
  FIN SI
FIN SI

// 3. Bucle para agregar productos a la venta
REPETIR
  MOSTRAR "----- Catálogo de Productos -----"
  imprimir_productos()  // Muestra la lista

  MOSTRAR "Ingrese ID del producto a vender:"
  LEER id_producto
  producto = buscar_producto(id_producto)

  SI producto es NULO ENTONCES
```

```
        MOSTRAR "Producto no encontrado."
    SINO
        MOSTRAR "Ingrese cantidad:"
        LEER cantidad

        SI cantidad <= producto.stock ENTONCES
            // Procesar el item
            subtotal = cantidad * producto.precio_venta
            total_venta = total_venta + subtotal

            // Agregar item a la lista de la venta
            AGREGAR A lista_productos_vendidos {producto_id: id_producto, cantidad: cantidad, subtotal: subtotal}

            // Actualizar stock (llamada a proceso de inventario)
            actualizar_stock(id_producto, cantidad, "restar")

            MOSTRAR "Producto agregado. Subtotal: ", subtotal
        SINO
            MOSTRAR "Stock insuficiente. Stock actual: ", producto.stock
        FIN SI
    FIN SI

    MOSTRAR "¿Agregar otro producto? (S/N)"
    LEER continuar
  HASTA QUE continuar == 'N'

  // 4. Seleccionar tipo de pago
  MOSTRAR "Total de la venta: ", total_venta
```

```
  MOSTRAR "Tipo de pago (1-Contado, 2-Crédito):"
  LEER tipo_pago

  SI tipo_pago == 2 ENTONCES  // Crédito
    deuda_actual = calcular_deuda_cliente(cliente.id)
    SI (deuda_actual + total_venta) <= cliente.limite_credito ENTONCES
      estado_venta = "Pendiente"
      MOSTRAR "Crédito autorizado."
    SINO
      MOSTRAR "Límite de crédito excedido. Venta no autorizada."
      // Revertir el stock (llamada a proceso de inventario)
      PARA CADA item EN lista_productos_vendidos HACER
        actualizar_stock(item.producto_id, item.cantidad, "sumar")
      FIN PARA
      TERMINAR
    FIN SI
  SINO
    estado_venta = "Pagado"
  FIN SI

  // 5. Guardar el registro de la venta
  id_venta = generar_id_venta()
  fecha_venta = fecha_actual()

  // Crear el registro de venta
  registro_venta = {id: id_venta, cliente_id: cliente.id, fecha: fecha_venta, total: total_venta, estado: estado_venta, productos: lista_productos_vendidos}
  AGREGAR registro_venta a la lista_global_ventas
```

```
  MOSTRAR "¡Venta registrada con éxito! ID: ", id_venta
FIN
```

Implementación básica en Python que sigue la estructura de los módulos definidas en el ejemplo. Usaremos listas y diccionarios como estructuras de datos en memoria (sin base de datos para mantenerlo simple y enfocado en la lógica) luego se puede agregar la pertinencia con JSON / CSV.

```
import datetime
from tabulate import tabulate  # Para tablas bonitas, instalar con: pip install tabulate

# --- DATOS GLOBALES (en memoria) ---
clientes = []
proveedores = []
productos = []
ventas = []
ordenes_compra = []

# --- MÓDULO 1: ENTRADA DE DATOS (Submódulos) ---

# A.1 Gestión de Clientes
def alta_cliente():
    print("\n--- REGISTRO DE NUEVO CLIENTE ---")
    id_cliente = input("ID del cliente: ")
    nombre = input("Nombre: ")
    telefono = input("Teléfono: ")
    direccion = input("Dirección: ")
    limite_credito = float(input("Límite de crédito: "))
    cliente = {
        "id": id_cliente, "nombre": nombre, "telefono": telefono,
        "direccion": direccion, "limite_credito": limite_credito
    }
    clientes.append(cliente)
    print("Cliente registrado con éxito.")
    return cliente

def buscar_cliente(id_cliente):
```

```
    for c in clientes:
        if c["id"] == id_cliente:
            return c
    return None

# A.2 Gestión de Proveedores
def alta_proveedor():
    print("\n--- REGISTRO DE NUEVO PROVEEDOR ---")
    id_prov = input("ID del proveedor: ")
    nombre = input("Nombre: ")
    telefono = input("Teléfono: ")
    direccion = input("Dirección: ")
    proveedor = {"id": id_prov, "nombre": nombre, "telefono": telefono, "direccion":
direccion}
    proveedores.append(proveedor)
    print("Proveedor registrado con éxito.")
    return proveedor

# A.3 Gestión de Productos
def alta_producto():
    print("\n--- REGISTRO DE NUEVO PRODUCTO ---")
    id_prod = input("ID del producto: ")
    nombre = input("Nombre: ")
    categoria = input("Categoría (ej. Lácteo, Grano, Verdura): ")
    precio_compra = float(input("Precio de compra: "))
    precio_venta = float(input("Precio de venta: "))
    fecha_caducidad = input("Fecha de caducidad (AAAA-MM-DD): ")
    stock_inicial = int(input("Stock inicial: "))
    producto = {
        "id": id_prod, "nombre": nombre, "categoria": categoria,
        "precio_compra": precio_compra, "precio_venta": precio_venta,
        "fecha_caducidad": fecha_caducidad, "stock": stock_inicial
    }
    productos.append(producto)
```

```
    print("Producto registrado con éxito.")
    return producto

def buscar_producto(id_producto):
    for p in productos:
        if p["id"] == id_producto:
            return p
    return None

# A.4 Gestión de Ventas
def generar_id_venta():
    return f"V-{datetime.datetime.now().strftime('%Y%m%d%H%M%S')}"

def registrar_venta():
    print("\n--- REGISTRO DE VENTA ---")
    total_venta = 0.0
    productos_vendidos = []

    # 1. Identificar cliente
    id_cliente = input("ID del cliente: ")
    cliente = buscar_cliente(id_cliente)
    if cliente is None:
        print("Cliente no encontrado.")
        resp = input("¿Desea registrarlo? (S/N): ").upper()
        if resp == 'S':
            cliente = alta_cliente()
        else:
            print("Venta cancelada.")
            return

    # 2. Bucle para agregar productos
    while True:
        print("\n--- CATÁLOGO DE PRODUCTOS ---")
        imprimir_productos()  # Llamada a módulo de salida
```

```
        id_prod = input("ID del producto a vender (o 'F' para finalizar): ")
        if id_prod.upper() == 'F':
            break

        producto = buscar_producto(id_prod)
        if producto is None:
            print("Producto no encontrado.")
            continue

        cantidad = int(input("Cantidad: "))
        if cantidad > producto["stock"]:
            print(f"Stock insuficiente. Solo hay {producto['stock']} unidades.")
            continue

        subtotal = cantidad * producto["precio_venta"]
        total_venta += subtotal
        productos_vendidos.append({"producto_id": id_prod, "cantidad": cantidad,
"subtotal": subtotal})

        # Actualizar stock (llamada a proceso)
        actualizar_stock(id_prod, cantidad, "restar")
        print(f"Producto agregado. Subtotal: ${subtotal:.2f}")

    if not productos_vendidos:
        print("No se agregaron productos. Venta cancelada.")
        return

    # 3. Tipo de pago
    print(f"\nTotal de la venta: ${total_venta:.2f}")
    tipo_pago = input("Tipo de pago (1-Contado, 2-Crédito): ")

    estado = "Pagado"
    if tipo_pago == "2":
```

```
        deuda_actual = calcular_deuda_cliente(cliente["id"])
        if (deuda_actual + total_venta) <= cliente["limite_credito"]:
            estado = "Pendiente"
            print("Crédito autorizado.")
        else:
            print("Límite de crédito excedido. Venta no autorizada.")
            # Revertir stock
            for item in productos_vendidos:
                actualizar_stock(item["producto_id"], item["cantidad"], "sumar")
            return

    # 4. Guardar venta
    id_venta = generar_id_venta()
    fecha_venta = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    venta = {
        "id": id_venta, "cliente_id": cliente["id"], "fecha": fecha_venta,
        "total": total_venta, "estado": estado, "productos": productos_vendidos
    }
    ventas.append(venta)
    print(f"¡Venta registrada con éxito! ID: {id_venta}")

# A.5 Gestión de Órdenes de Compra
def registrar_orden_compra():
    print("\n--- REGISTRO DE ORDEN DE COMPRA ---")
    id_orden = f"OC-{datetime.datetime.now().strftime('%Y%m%d%H%M%S')}"
    id_prov = input("ID del proveedor: ")
    proveedor = None
    for p in proveedores:
        if p["id"] == id_prov:
            proveedor = p
            break
    if proveedor is None:
        print("Proveedor no encontrado.")
        return
```

```
    productos_orden = []
    while True:
        id_prod = input("ID del producto a ordenar (o 'F' para finalizar): ")
        if id_prod.upper() == 'F':
            break
        producto = buscar_producto(id_prod)
        if producto is None:
            print("Producto no encontrado.")
            continue
        cantidad = int(input("Cantidad: "))
        productos_orden.append({"producto_id": id_prod, "cantidad": cantidad})
        # Actualizar stock (entrada de mercancía)
        actualizar_stock(id_prod, cantidad, "sumar")

    if not productos_orden:
        print("No se agregaron productos. Orden cancelada.")
        return

    fecha_pedido = datetime.datetime.now().strftime("%Y-%m-%d")
    fecha_entrega = input("Fecha de entrega estimada (AAAA-MM-DD): ")
    orden = {
        "id": id_orden, "proveedor_id": id_prov, "fecha_pedido": fecha_pedido,
        "fecha_entrega": fecha_entrega, "productos": productos_orden
    }
    ordenes_compra.append(orden)
    print(f"Orden de compra registrada con éxito. ID: {id_orden}")

# --- MÓDULO 2: PROCESAMIENTO ---

# B.1 Control de Inventario
def actualizar_stock(producto_id, cantidad, operacion):
    producto = buscar_producto(producto_id)
    if producto:
```

```
        if operacion == "restar":
            producto["stock"] -= cantidad
        elif operacion == "sumar":
            producto["stock"] += cantidad
        print(f"Stock de '{producto['nombre']}' actualizado. Nuevo stock: {producto['stock']}")
    else:
        print(f"Producto con ID {producto_id} no encontrado para actualizar stock.")

# B.2 Gestión de Créditos
def calcular_deuda_cliente(cliente_id):
    deuda = 0.0
    for venta in ventas:
        if venta["cliente_id"] == cliente_id and venta["estado"] == "Pendiente":
            deuda += venta["total"]
    return deuda

# B.3 Motor de Reportes
def calcular_ventas_por_periodo(fecha_inicio, fecha_fin):
    total = 0.0
    for venta in ventas:
        fecha_venta = datetime.datetime.strptime(venta["fecha"], "%Y-%m-%d %H:%M:%S").date()
        if fecha_inicio <= fecha_venta <= fecha_fin:
            total += venta["total"]
    return total

def calcular_top_10_productos():
    ventas_por_producto = {}
    for venta in ventas:
        for item in venta["productos"]:
            prod_id = item["producto_id"]
            cantidad = item["cantidad"]
            if prod_id in ventas_por_producto:
                ventas_por_producto[prod_id] += cantidad
```

```
            else:
                ventas_por_producto[prod_id] = cantidad

    # Ordenar por cantidad vendida (de mayor a menor) y tomar los 10 primeros
    top_10 = sorted(ventas_por_producto.items(), key=lambda x: x[1], reverse=True)[:10]

    # Obtener nombres de productos
    resultado = []
    for prod_id, cantidad in top_10:
        producto = buscar_producto(prod_id)
        if producto:
            resultado.append({"Nombre": producto["nombre"], "Cantidad Vendida": cantidad})
        else:
            resultado.append({"Nombre": prod_id, "Cantidad Vendida": cantidad})  # Fallback
    return resultado

# --- MÓDULO 3: SALIDA DE DATOS ---

# C.1 Generador de Listados
def imprimir_clientes():
    print("\n--- LISTA DE CLIENTES ---")
    if not clientes:
        print("No hay clientes registrados.")
        return
    headers = ["ID", "Nombre", "Teléfono", "Dirección", "Límite de Crédito", "Deuda Actual"]
    data = []
    for c in clientes:
        deuda = calcular_deuda_cliente(c["id"])
        data.append([c["id"],      c["nombre"],      c["telefono"],      c["direccion"],
f"${c['limite_credito']:.2f}", f"${deuda:.2f}"])
    print(tabulate(data, headers=headers, tablefmt="grid"))

def imprimir_proveedores():
    print("\n--- LISTA DE PROVEEDORES ---")
```

```
    if not proveedores:
        print("No hay proveedores registrados.")
        return
    headers = ["ID", "Nombre", "Teléfono", "Dirección"]
    data = [[p["id"], p["nombre"], p["telefono"], p["direccion"]] for p in proveedores]
    print(tabulate(data, headers=headers, tablefmt="grid"))


def imprimir_productos():
    print("\n--- CATÁLOGO DE PRODUCTOS ---")
    if not productos:
        print("No hay productos registrados.")
        return
    headers = ["ID", "Nombre", "Categoría", "Precio Compra", "Precio Venta", "Caducidad",
"Stock"]
    data = []
    for p in productos:
        data.append([p["id"], p["nombre"], p["categoria"], f"${p['precio_compra']:.2f}",
f"${p['precio_venta']:.2f}", p["fecha_caducidad"], p["stock"]])
    print(tabulate(data, headers=headers, tablefmt="grid"))


# C.2 Generador de Reportes de Inventario
def productos_por_vencer(dias=7):
    print(f"\n--- PRODUCTOS POR VENCER (próximos {dias} días) ---")
    hoy = datetime.date.today()
    limite = hoy + datetime.timedelta(days=dias)
    encontrados = False
    for p in productos:
        fecha_cad = datetime.datetime.strptime(p["fecha_caducidad"], "%Y-%m-%d").date()
        if hoy <= fecha_cad <= limite:
            print(f"  - {p['nombre']} (ID: {p['id']}) - Caduca: {fecha_cad} - Stock:
{p['stock']}")
            encontrados = True
    if not encontrados:
        print("No hay productos por vencer en este período.")
```

```
# C.3 Generador de Reportes Financieros
def imprimir_clientes_con_credito():
    print("\n--- CLIENTES CON CRÉDITO PENDIENTE ---")
    encontrados = False
    for c in clientes:
        deuda = calcular_deuda_cliente(c["id"])
        if deuda > 0:
            print(f" - {c['nombre']} (ID: {c['id']}) - Deuda: ${deuda:.2f} (Límite: ${c['limite_credito']:.2f})")
            encontrados = True
    if not encontrados:
        print("No hay clientes con deuda pendiente.")

def imprimir_ventas_por_periodo():
    print("\n--- REPORTE DE VENTAS POR PERÍODO ---")
    fecha_ini_str = input("Fecha de inicio (AAAA-MM-DD): ")
    fecha_fin_str = input("Fecha de fin (AAAA-MM-DD): ")
    try:
        fecha_ini = datetime.datetime.strptime(fecha_ini_str, "%Y-%m-%d").date()
        fecha_fin = datetime.datetime.strptime(fecha_fin_str, "%Y-%m-%d").date()
        total = calcular_ventas_por_periodo(fecha_ini, fecha_fin)
        print(f"Total de ventas entre {fecha_ini_str} y {fecha_fin_str}: ${total:.2f}")
    except ValueError:
        print("Formato de fecha incorrecto. Use AAAA-MM-DD.")

def imprimir_ordenes_compra():
    print("\n--- LISTADO DE ÓRDENES DE COMPRA ---")
    if not ordenes_compra:
        print("No hay órdenes de compra registradas.")
        return
    headers = ["ID Orden", "Proveedor", "Fecha Pedido", "Fecha Entrega", "Productos"]
    data = []
    for oc in ordenes_compra:
        proveedor = "No encontrado"
```

```
        for p in proveedores:
            if p["id"] == oc["proveedor_id"]:
                proveedor = p["nombre"]
                break
        productos_str = ""
        for item in oc["productos"]:
            prod = buscar_producto(item["producto_id"])
            nombre_prod = prod["nombre"] if prod else item["producto_id"]
            productos_str += f"{nombre_prod} (x{item['cantidad']}), "
        data.append([oc["id"],   proveedor,   oc["fecha_pedido"],   oc["fecha_entrega"],
productos_str.rstrip(", ")])
    print(tabulate(data, headers=headers, tablefmt="grid"))


def imprimir_top_10():
    print("\n--- TOP 10 PRODUCTOS MÁS VENDIDOS ---")
    top = calcular_top_10_productos()
    if not top:
        print("No hay ventas registradas.")
        return
    print(tabulate(top, headers="keys", tablefmt="grid"))


# --- MENÚ PRINCIPAL ---
def menu_principal():
    while True:
        print("\n" + "="*40)
        print("   SISTEMA DE GESTIÓN 'LA VENTECITA'")
        print("="*40)
        print("1. Registrar Cliente")
        print("2. Registrar Proveedor")
        print("3. Registrar Producto")
        print("4. Registrar Venta")
        print("5. Registrar Orden de Compra")
        print("6. Ver Lista de Clientes")
        print("7. Ver Lista de Proveedores")
```

```
print("8. Ver Catálogo de Productos")
print("9. Ver Productos por Vencer")
print("10. Ver Clientes con Crédito")
print("11. Ver Ventas por Período")
print("12. Ver Órdenes de Compra")
print("13. Ver Top 10 Productos Más Vendidos")
print("0. Salir")
print("-"*40)

opcion = input("Seleccione una opción: ")

if opcion == "1":
    alta_cliente()
elif opcion == "2":
    alta_proveedor()
elif opcion == "3":
    alta_producto()
elif opcion == "4":
    registrar_venta()
elif opcion == "5":
    registrar_orden_compra()
elif opcion == "6":
    imprimir_clientes()
elif opcion == "7":
    imprimir_proveedores()
elif opcion == "8":
    imprimir_productos()
elif opcion == "9":
    productos_por_vencer()
elif opcion == "10":
    imprimir_clientes_con_credito()
elif opcion == "11":
    imprimir_ventas_por_periodo()
elif opcion == "12":
```

```
            imprimir_ordenes_compra()
        elif opcion == "13":
            imprimir_top_10()
        elif opcion == "0":
            print("¡Gracias por usar el sistema! Hasta luego.")
            break
        else:
            print("Opción no válida. Intente de nuevo.")

        input("\nPresione Enter para continuar...")

# --- EJECUCIÓN DEL PROGRAMA ---
if __name__ == "__main__":
    # Datos de ejemplo para iniciar
    clientes.append({"id": "C001", "nombre": "María López", "telefono": "555-1234", "direccion": "Calle 1", "limite_credito": 500.0})
    clientes.append({"id": "C002", "nombre": "Juan Pérez", "telefono": "555-5678", "direccion": "Calle 2", "limite_credito": 300.0})
    proveedores.append({"id": "P001", "nombre": "Lácteos S.A.", "telefono": "555-9012", "direccion": "Zona Industrial"})
    productos.append({"id": "PROD1", "nombre": "Queso Fresco", "categoria": "Lácteo", "precio_compra": 3.0, "precio_venta": 5.0, "fecha_caducidad": "2026-08-25", "stock": 20})
    productos.append({"id": "PROD2", "nombre": "Leche Entera", "categoria": "Lácteo", "precio_compra": 1.0, "precio_venta": 1.8, "fecha_caducidad": "2026-08-22", "stock": 50})
    productos.append({"id": "PROD3", "nombre": "Frijoles Negros", "categoria": "Grano", "precio_compra": 1.5, "precio_venta": 2.5, "fecha_caducidad": "2027-01-01", "stock": 100})

    menu_principal()
```